Report - AIML Hackathon 25/26

Team AMA: Giovanni Adelfio, Elena Mannoni, Michele Aliffi **16/12/2026**

To address the passage ranking challenge, we implemented a multi-stage *Learning to Rank* (LTR) pipeline. Our approach decomposes the problem into three distinct phases: candidate generation, feature engineering, and final re-ranking.

1 Hybrid Retrieval (Candidate Generation)

We adopted a hybrid retrieval strategy to efficiently filter the collection of over 650,000 passages down to a manageable set of high-potential candidates. For each query, we retrieved the top-100 passages using a combination of:

- **BM25:** Implemented via the `bm25_pt` library on GPU to capture exact keyword matches;
- **Cosine Similarity:** Calculated between query and passage embeddings (`ms-marco-MiniLM-L-6-v2` model) to capture semantic meaning.

The final candidate pool was created by merging the top results from both retrieval methods, ensuring high recall by covering both keyword-based and semantic nuances.

2 Feature Engineering: Cross Encoder and Lexical/Structural Analysis

The selected candidates were processed using a Cross-Encoder model to capture deeper relevance signals. We utilized the `cross-encoder/ms-marco-MiniLM-L-6-v2` model. Unlike bi-encoders, this model processes the query and passage simultaneously, allowing the self-attention mechanism to identify complex relationships between terms. This step generated a `cross_score`, which served as the strongest predictor of relevance in our pipeline.

In addition to the semantic score, we engineered a comprehensive set of features for each query-passage pair to train our ranking model. These features include:

- **Structural features**: query length, passage length, first match position;
- **Lexical overlap**: overlap count, overlap ratio, bigram overlap, LCS score, IDF weighted overlap;
- **Domain specific**: a boolean `has_code_match` feature to specifically target technical queries containing code snippets.

3 Final Re-Rank (LightGBM)

For the final ranking, we trained a **LightGBM** model. We treated the task as a pure ranking problem, using the validation set to monitor performance (optimized for MAP@10) and applying early stopping to prevent overfitting. The model aggregates the dense signals from the Cross-Encoder with the sparse lexical signals to produce the optimal permutation of the candidate list.

We experimented with different configurations, including a larger 12-layer Cross-Encoder and varying feature sets. We found that a concise feature set consisting of (`cross`, `bm25_score`, `overlap_ratio`, `cosine_sim`) combined with the 6-layer MiniLM model yielded the best generalization.

Conclusion

Our hybrid approach demonstrated superior performance compared to single-method baselines. The integration of the **Cross-Encoder** within a **Gradient Boosting** framework proved decisive, allowing us to leverage the semantic understanding of pre-trained language models while retaining explicit lexical signals for precision. To develop these systems both **Claude Sonnet** and **Gemini 3 Pro** LLMs were used.